

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 4 — Finire e confrontare

Ripresa e confronto

Dove era arrivato il laboratorio

La seconda parte: finire, verificare, confrontare

- Il laboratorio del Modulo 4: Claude Code al lavoro in un repository, una **traccia** a scelta fra tre, sui progetti sintetici di CiaspoLink — il gestionale PHP e la rete. Nella prima parte, il lavoro; in questa, la **seconda**, si finisce, si verifica e si confronta
- a **M4-E02, una funzione nuova, con i test**: il filtro dei ticket del gestionale per apparato e per intervallo di date, da `SPECIFICA.md`. I test li scrive un subagent; un revisore guarda il diff senza aver visto scrivere il codice
- b **M4-E03, un modulo opaco**: `legacy/chiusura_mese.inc.php`, mappato da due subagent Explore, documentato e fissato con i test di caratterizzazione; in questa parte, la **proposta** di refactoring, in plan mode. Il modulo non si tocca
- c **Uno script operativo che gira davvero**, con un revisore che legge e non modifica, in due varianti: **M4-E04**, il controllo notturno del gestionale, in PHP; **M4-E05**, il rapporto quotidiano dei disservizi dai log della rete, in Python o in Bash
- Le regole: prima il **piano**; almeno un **subagent**; un **controllo** che Claude può eseguire; piccoli passi, un **commit** ciascuno; Sonnet, effort medium
- La prima parte si è chiusa con un commit e `NOTA.md`: che cosa è fatto, che cosa manca, i comandi per riprendere. Da lì si riparte: `NOTA.md` e `git log` vi dicono **dove eravate**

Riprendere: dalla nota o dalla conversazione

Una sessione nuova che legge la nota, oppure la conversazione di prima

```
cd ~/corso-claude-code/M4-E02   # o M4-E03, M4-E04, M4-E05
git log --oneline               # i commit della prima parte
claude --model sonnet --effort medium
# sessione nuova: il prompt 5 del README, che legge NOTA.md
# oppure la conversazione di prima: claude --continue
```

- Una sessione nuova parte **senza la conversazione** di prima: le sessioni sono indipendenti. Non da zero: `CLAUDE.md` e la memoria automatica (auto memory) si caricano prima ancora che scriviate. Ciò che serve del lavoro lo porta la **nota**: il prompt 5 di ogni traccia la fa leggere **per prima**. Nella traccia (b), prima, plan mode: Shift+Tab, o `/plan` davanti al prompt
- Oppure `claude --continue`, o `-c`: riprende la conversazione **più recente** della cartella in cui siete; se lì non ce n'è una, stampa «No conversation found to continue» ed esce. Tornano messaggi, chiamate agli strumenti e risultati; **non** i comandi Bash in background, né opzioni come `--mcp-config` o `--settings`, da ripassare
- Il prezzo: dopo un'interruzione abbastanza lunga, la richiesta successiva **ricalcola tutto l'input** e ricostruisce la cache; sul Pro, una conversazione lunga ripresa dopo una lunga interruzione può far comparire il dialogo «Resume this conversation?»
- A voce: che cosa vi manca per finire?

Una nota scritta vale più di una conversazione lunga da ricaricare.

Finire

Che cosa resta, traccia per traccia, e la verifica

- **a** **M4-E02**: i requisiti di `SPECIFICA.md` che mancano, un passo alla volta, un commit per passo; dopo ogni passo il controllo, la suite PHPUnit del gestionale; prima dell'ultimo commit, il diff al **revisore**, poi le sue correzioni, di nuovo il controllo e, quando passa, l'ultimo commit
- **b** **M4-E03**: in plan mode, la **proposta** di refactoring, `PROPOSTA.md`: passi numerati, ciascuno con la sua riga «Controllo:»; approvato il piano, il controllo del primo passo e il commit
- **c** **M4-E04** e **M4-E05**: ciò che manca dello script, con il controllo dopo ogni passo — `python3 controlla_notturno.py` o `python3 controlla_rapporto.py` — finché l'ultima riga dice `uscita: 0`; prima dell'ultimo commit, il diff al **revisore**, poi le sue correzioni, di nuovo il controllo e, quando passa, l'ultimo commit
- ☑ Poi `/exit`, `cd ~/esercitazioni` — o il percorso in cui avete clonato il repository — e `tools/verifica.sh`, con l'identificativo della vostra traccia
- 🛟 Se qualcosa non va, dice la pagina delle best practice di Claude Code, è **quasi sempre** una di quattro situazioni:
  1. ha costruito **altro**: ditegli ciò che non vede
  2. dice di aver **finito**, e non è così: dategli un controllo da eseguire
  3. ha fatto ciò che **non deve**, o saltato ciò che deve: imponetelo, con regole e hook
  4. troppo **tempo**, token o richieste di permesso: una **modalità** di permesso decisa una volta; il modello e l'effort, che pesano sui limiti; `/usage`, che mostra che cosa ha pesato

Al lavoro

M4-E02 · M4-E03 · M4-E04 · M4-E05: finire, e far passare la verifica

- I passi sono nella **seconda parte** del README della vostra traccia, nel repository delle esercitazioni: `m4_claude_code_agenti/s3_agente_nel_repository/m4_e2_laboratorio/`, una cartella per traccia
- Traguardi: la sessione riparte dalla **nota**; il **controllo** è verde; il diff è passato al **revisore** — nella traccia (b), il **piano** è approvato; l'ultimo **commit**; `tools/verifica.sh` esce con **0**
- Dai README, «Se qualcosa va storto»: una riga che unisce più comandi? **No**, e chiedete un comando per volta. `claude --continue` risponde «No conversation found to continue»? Nella cartella in cui siete non c'è una conversazione da riprendere: entrate nello **spazio di lavoro**
- L'impronta: Sonnet, effort medium; i controlli e la verifica girano sul vostro computer, **senza chiamate** al modello; `/usage` vi dice quanto avete usato del piano

La verifica è passata?

A voce, uno per uno: la traccia, e l'uscita di `tools/verifica.sh`

- A voce, ciascuno: la vostra **traccia** — per la (c), la variante — e l'**uscita** della verifica: **0** fatto; **1** non ancora, e dice che cosa manca; **2** non può partire

traccia	la verifica esce con 0 quando
(a) M4-E02	la suite intera e i test nuovi passano sul vostro codice, e i test nuovi **falliscono** su quello di partenza
(b) M4-E03	documentazione e proposta sono complete; i test di caratterizzazione passano sul modulo com'era e **falliscono** su una copia rotta apposta; `legacy/` è com'era; i file che i passi chiedono sono nei **commit**
(c) M4-E04	il controllo dell'esercizio passa; lo script trova le incoerenze dei dati di esempio, e **non ne trova** dove non ci sono
(c) M4-E05	il controllo dell'esercizio passa; lo script trova il disservizio che la verifica conosce, campo per campo, e in un giorno tranquillo **tace**

- Nelle tracce (a) e (c) esce con 1 anche se il **revisore** ha modificato file: con soli `Read`, `Grep` e `Glob` non può, e il limite lo impone Claude Code
- Un 1 non è un voto: dice **che cosa manca**, ed è il punto da cui ripartire

Il subagent e il piano

Che cosa ha fatto il subagent, e che cosa ha cambiato il piano

- A voce: che cosa ha fatto il vostro **subagent**, e che cosa vi ha riportato? Ha trovato qualcosa che la sessione principale **non aveva visto**? E il **piano**: che cosa avete cambiato, leggendolo, prima di approvarlo?
- I subagent delle tracce: in (a) `autore-test`, che scrive i test dalla specifica, e il `revisore`; in (b) due **Explore**, in parallelo, per la mappa; in (c) il `revisore`. Danno parallelismo o uno **sguardo fresco**, non una risposta più intelligente
- La trascrizione di un subagent è un **file suo**: in `~/.claude/projects/`, nella cartella della sessione, sotto `subagents/`, un `agent-{agentId}.jsonl` per subagent
- La verifica conta le chiamate al tool **Agent**, per tipo di subagent: come **dato**, mai come esito, perché il formato di quei file è interno a Claude Code e cambia fra le versioni. Nelle tracce (a) e (c) cerca anche, nella trascrizione del revisore, gli strumenti che **modificano**
- I piani di plan mode stanno in `piani/`, nello spazio di lavoro: lì rileggete che cosa avevate **approvato**

Che cosa si è rotto

E come ne siete usciti: le vie d'uscita di Claude Code, e dei README

- A voce: che cosa si è **rotto**, nel vostro laboratorio, e come ne siete **usciti**?
- **Esc**, appena Claude devia: lo ferma a metà azione e **conserva** il lavoro fatto fino lì
- Dopo **due correzioni fallite**, non tentate la terza: `/clear` e un prompt migliore, che metta dentro ciò che avete imparato
- `/rewind`, o **Esc due volte** a prompt vuoto: si torna a un punto precedente, con codice e conversazione insieme, con la sola conversazione o con il solo codice. Un file cambiato **solo da comandi Bash** non torna: per quello, **git**
- Dai README: una regola deny che ferma una modifica è **la regola che funziona**. Si corregge il codice, o il test; se la specifica vi sembra sbagliata, ditelo al docente
- Per ricominciare **da capo**: `tools/reset_esercizio.sh`, con l'identificativo della traccia

Quanto è costato

/usage, subagent per subagent; e che cosa rifareste

- /usage: il costo della sessione, i limiti del piano, le statistiche; sul Pro divide l'utilizzo per skill, **subagent**, plugin e server MCP. Quanto hanno pesato i vostri subagent?
- I subagent costano soprattutto **token**: ognuno manda le sue richieste, che contano sullo stesso utilizzo della conversazione principale. Per spendere meno: **risposte brevi**, e **Haiku** dove il lavoro è semplice, come quello del revisore
- I limiti sono gli **stessi di claude.ai**: tutta l'attività nei due strumenti consuma gli stessi limiti
- A voce: che cosa **rifareste diversamente**?

Un controllo nel codice dice se il lavoro è fatto; voi decidete se è quello che serviva.